

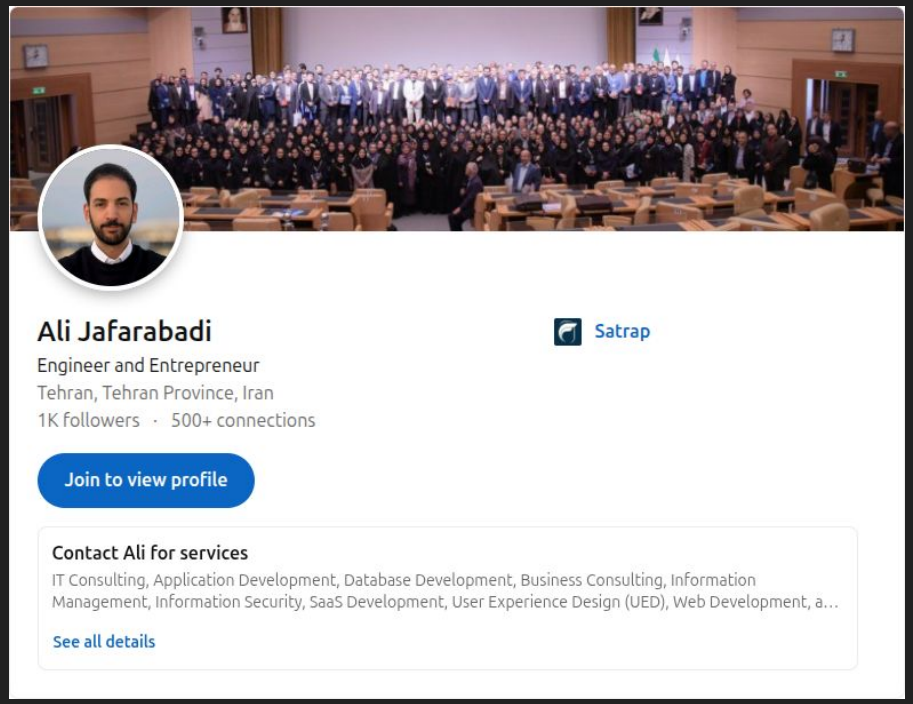


C++ Reimagined

“Rediscovering the Language through C++11 to C++26”

October 2026 - 15th Linux Fest, Amirkabir University of Technology

Ali Jafarabadi



@alij8i - jafarabadi@live.com

- Co-Founder and CTO, Satrap Inc.
- Member of AI and Data Science Committee of 5th NSR Organization
- Member of Libre/Free Knowledge Foundation
- Project Manager for the Smart Transformation of the:
  - Tehran Heart Center
  - Iran's Aviation Industry
  - Securities and Exchange Organization of Iran
  - Geographical Organization of Iran

Agenda

1. Why rediscover C++ now
2. Origins: C with Classes and the pre-standard era
3. C++98 and C++03 — the first contract
4. C++11, C++14, C++17 — the renaissance
5. C++20 and C++23 — the leap
6. C++26 — the frontier
7. How to rediscover the language

Why Rediscover C++ Now?

A different language

Value semantics, RAII, and move semantics replaced raw pointers and hand-written copy logic.

Safety without overhead

concepts, ranges, constexpr, and modules give compile-time guarantees with zero runtime cost.

Expressiveness

optional, variant, string_view, span, and print replace hand-rolled utility code.

Still zero-overhead

Nothing new abandons the original promise: C++ compiles to native code you can reason about.

If you learned C++ before 2011, you learned a different language.

Origins: C with Classes

01

1979 — 'C with Classes'

Stroustrup at Bell Labs: Simula's abstraction with C's efficiency.

02

One idea drove it

Classes as 'a better C', without giving up pointers or speed.

03

Two design goals

Data abstraction, while staying as close to C as the idea allows.

04

1983 — the name changes

Renamed C++; the compiler is Cfront. A language, not an extension of C.

The Pre-Standard Era

Early releases

1

Cfront releases: virtual functions, references, multiple inheritance.

Templates arrive early

2

Templates and the nascent STL arrive in Cfront 3.x.

Exceptions and RTTI

3

Exceptions, RTTI, and the ARM document what actually exists.

Namespaces appear

4

Namespaces arrive as the language goes multi-vendor.

C++98: The First Standard

One document gave every compiler the same contract: templates, the STL, exceptions, namespaces. Portability now meant 'standard C++', not 'whatever this compiler accepts'.

ISO/IEC 14882:1998

C++98 Feature Highlights

Core language

- Templates — generic programming becomes standard
- Exceptions and RTTI (dynamic_cast, typeid)
- Namespaces, bool, wchar_t, long long (in C++11 practice)
- New cast operators: static_cast, dynamic_cast, const_cast, reinterpret_cast
- mutable, explicit, and more precise const semantics

Standard library

- The STL: containers, iterators, algorithms, functors
- string, iostreams, and locales
- std::auto_ptr (a warning from the future — later deprecated)
- std::pair, vector, map, set, list, deque
- Note: export was specified but rarely implemented

C++03 Highlights: A Corrigendum, Not a Feature Release

Narrative: A Bug-Fix Release

- Purely a Technical Corrigendum: defect fixes, not new features
- Adopted TC1/TR1-era library extensions informally (smart pointers, regex)
- Stabilized the standard so compilers could target a well-defined baseline
- Set the stage for the long gap before C++11

C++03 Highlights

- Value semantics fixes and library defect reports (no new core features)
- std::shared_ptr, std::weak_ptr, and std::make_shared via TR1 adoption
- Hash containers: std::unordered_map and std::unordered_set
- std::function, std::bind, std::tuple, and std::regex

The Long Gap and TR1

2003–2007: the pause

1

The next revision misses its 2000s deadline; evolving C++ proves hard.

TR1 (2005–2007)

2

TR1 ships smart pointers, hash containers, tuples, regex.

TR1 in practice

3

Vendors ship TR1 as std::tr1:: — the first 'modern' library.

Boost fills the gap

4

Boost becomes the proving ground for TR1 and later C++11.

C++11: The Renaissance

After 13 years, C++11 redefined the language: move semantics made expensive copies optional, lambdas and auto lowered friction, and threads entered the standard.

C++11 Feature Highlights

Library

- std::unique_ptr, shared_ptr, weak_ptr
- std::thread, mutex, future, atomic
- std::chrono
- unordered_map / unordered_set
- std::tuple, std::regex, type traits

Language

- rvalue references and move semantics
- auto and decltype
- Lambdas; range-based for
- constexpr (limited); nullptr
- Uniform init, variadic templates, static_assert, override/final, enum class

C++14: Polish and Generic Lambdas

1

A rhythm forms

A small, deliberate release two years after C++11. It proved the committee could ship on a three-year train — refinement over revolution.

2

Finishing touches

Generic lambdas and relaxed constexpr moved C++11's ideas from 'usable' to 'comfortable' in everyday code.

3

The quiet headline

make_unique closes the last major gap in safe ownership — there is finally no reason to call new directly.

C++14 Feature Highlights

Library

- std::make_unique
- std::shared_timed_mutex
- std::integer_sequence
- std::quoted
- std::exchange; user-defined literals for chrono and string

Language

- Generic lambdas (auto parameters)
- Lambda init-capture
- Relaxed constexpr (loops, mutation)
- Return type deduction for functions
- Variable templates; binary literals; digit separators

C++17: Simplification and Vocabulary Types

C++17 removed friction — structured bindings, if constexpr, copy elision — and standardized the vocabulary types people hand-rolled for years: optional, variant, string_view, filesystem.

C++17 Feature Highlights

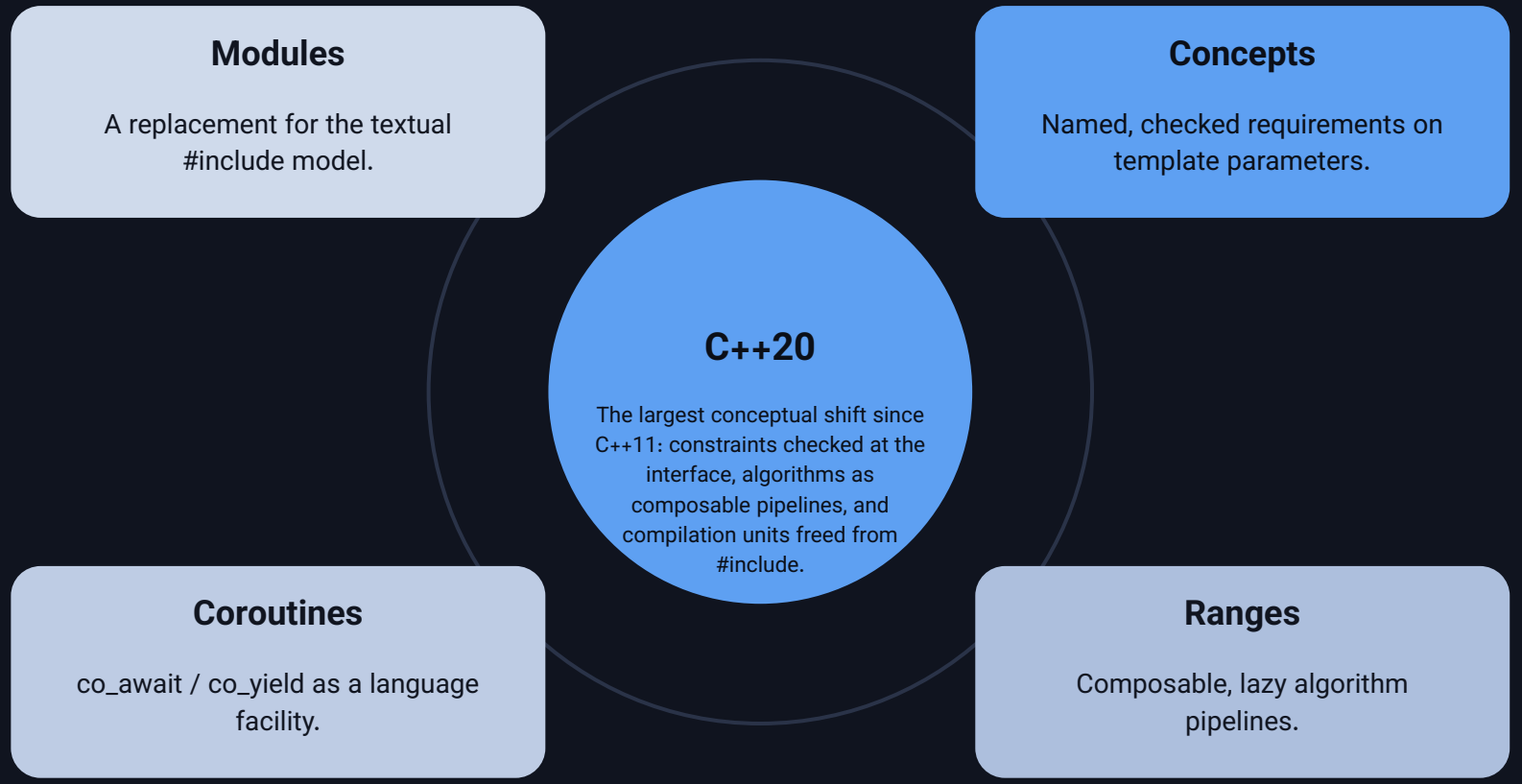
Language

- Structured bindings
- if constexpr and constexpr lambdas
- Fold expressions
- Class template argument deduction (CTAD)
- Inline variables; nested namespaces

Library

- std::optional, std::variant, std::any
- std::string_view
- std::filesystem
- Parallel algorithms and execution policies
- [[nodiscard]], [[maybe_unused]], [[fallthrough]]

C++20: The Big Four



C++20 Feature Highlights

Language

- Concepts and abbreviated function templates
- Ranges and views
- Coroutines (co_await, co_yield, co_return)
- Modules
- Three-way comparison (<=>); designated initializers

Library

- std::span
- std::format (shipped widely a bit later)
- std::jthread, stop_token
- std::source_location
- Calendar and time zones in <chrono>

ISO/IEC 14882:2024

C++23 Highlights: Ergonomics and Library Maturity

std::print

std::print and std::println bring formatted output to the standard library

Explicit object parameter

Deducing this (P0847) simplifies CRTP and ref-qualified member design

Multidimensional subscript

A library-wide effect that removes hazard: auto_ptr is finally gone

std::mdspan

Import std and C++20 module headers for faster builds

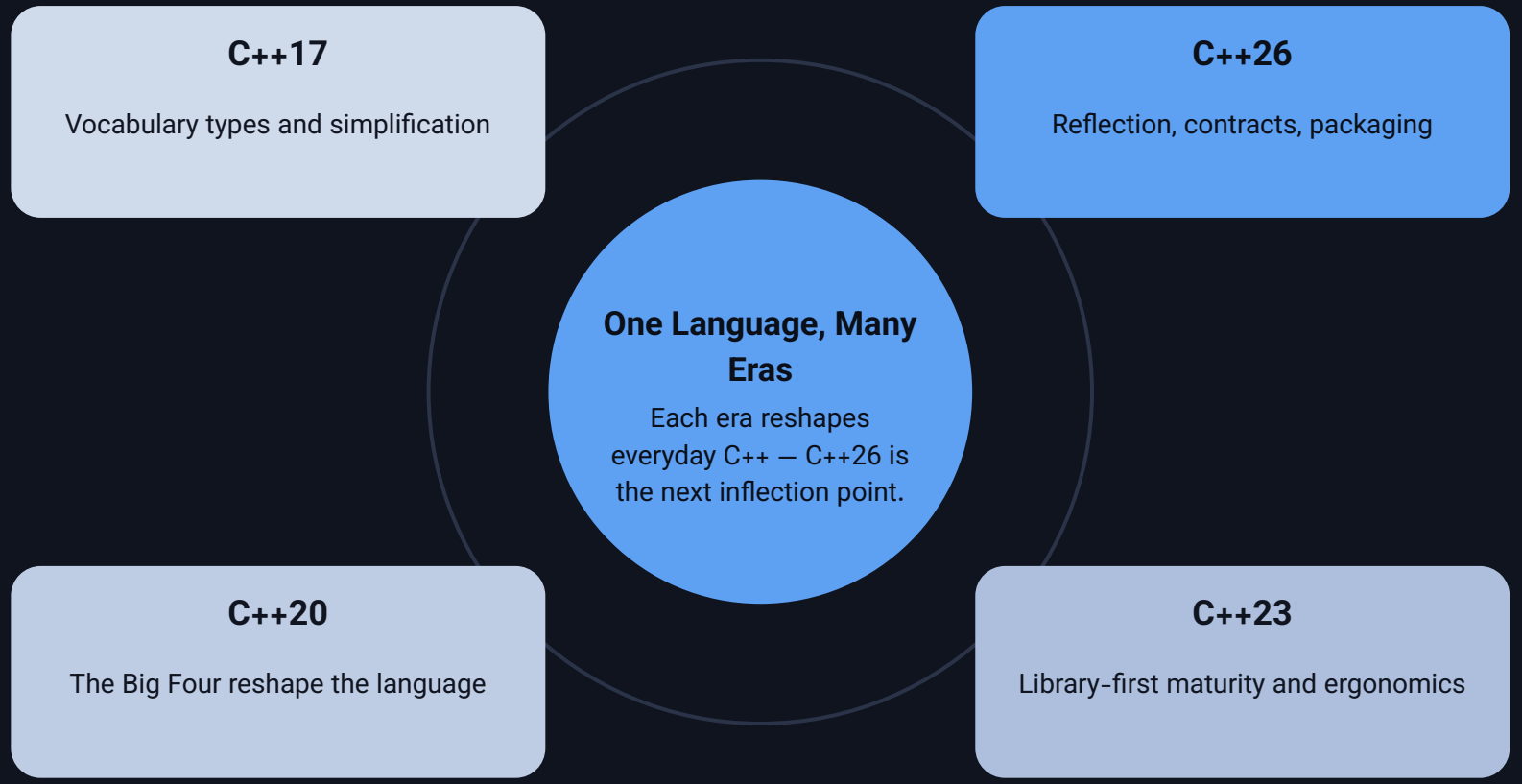
std::expected

std::stacktrace gives portable introspection into the call stack

Ranges & coroutine extras

Ranges::to, flat_map, generator coroutines, and move_only_function

C++26: The Draft Is Out — Support Is Still Landing



WG21 draft, voted out March 2026

C++26 Highlights: Reflection, Contracts, and Safety



Reflection

Static reflection via P2996 lets code query and generate types at compile time



Contracts

Language-level preconditions, postconditions, and assertions replace ad-hoc macros



Packaging & std::hazard

Standardized ecosystem groundwork and safer concurrency primitives



Profile & safety

Profiles aim to carve formally safer subsets of the language

C++ Reimagined: Your Roadmap from C++11 to C++26

Pick one feature per era. Start today.

Thank you for taking the time;

Ali Jafarabadi (@alij8i)